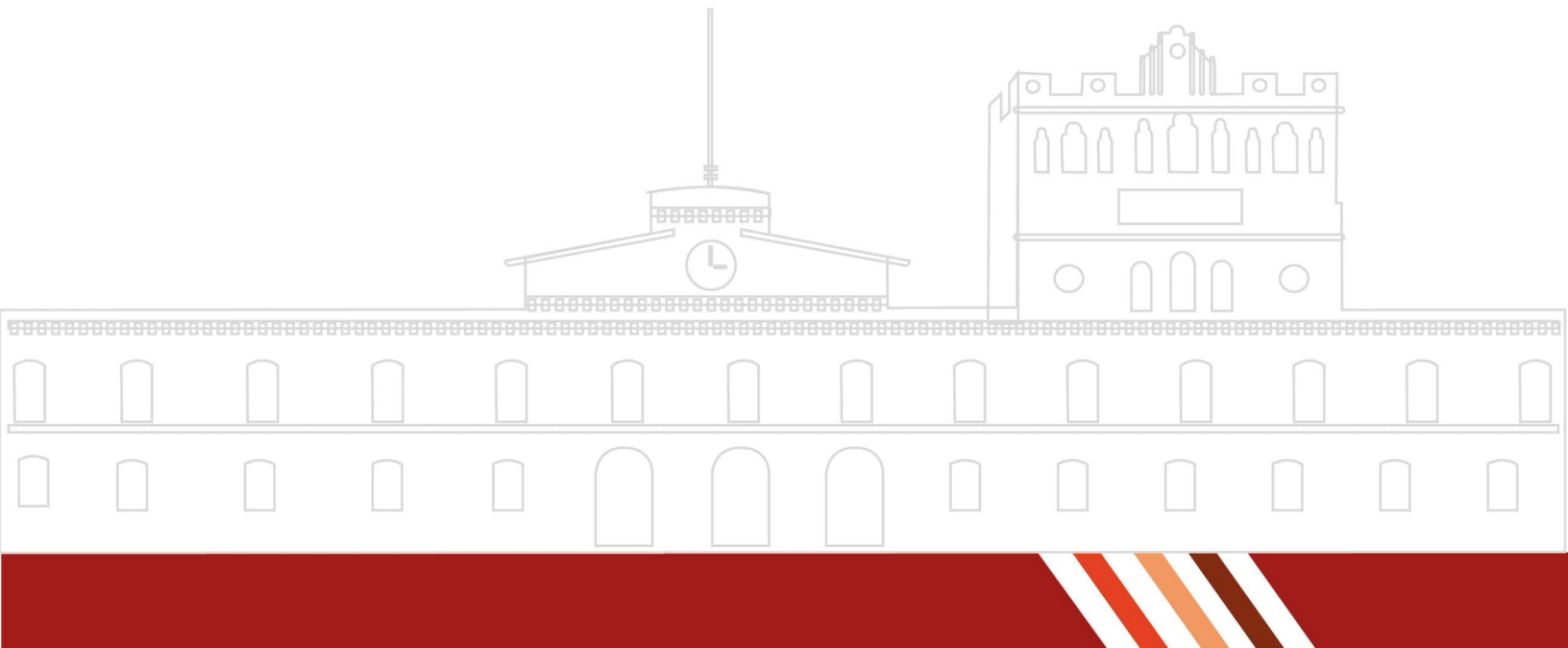


REPORTE DE PRÁCTICA NO. 8

Práctica 8 Análisis sintáctico de sentencias SQL

ALUMNOS:

Gutiérrez Reyes Jasson Axel
Pérez García Alejandro Máximo
Sánchez Barrón Javier Alejandro
Zúñiga Montoya José Antonio



Contents

1	Introducción	3
2	Marco Teórico	3
3	Objetivos	3
3.1	Objetivo General	3
3.2	Objetivos Específicos	3
4	Desarrollo	3
4.1	Librerías y Variables Externas	3
4.2	Compatibilidad de Formato (El Retorno de Windows)	4
4.3	Código del Analizador Léxico (lexico.l)	4
4.4	Código del Analizador Sintáctico (sintactico.y)	4
5	Resultados	6
6	Conclusiones	7
7	Cuestionario Semántico	7
8	Bibliografía	7

Introducción

El lenguaje de consulta estructurado (SQL) es el estándar universal para la gestión y manipulación de bases de datos relacionales. A diferencia de los lenguajes de propósito general, SQL es un lenguaje declarativo donde el usuario especifica qué datos desea obtener sin describir el algoritmo exacto para su extracción. El desarrollo de un analizador sintáctico para SQL permite validar que las consultas enviadas al motor de base de datos cumplan con las reglas gramaticales del estándar, asegurando la integridad de las operaciones de lectura y escritura.

Marco Teórico

El análisis sintáctico de SQL se basa en gramáticas de contexto libre que definen la estructura de las sentencias CRUD (Create, Read, Update, Delete). En esta práctica, nos enfocamos en las cuatro operaciones fundamentales:

- **SELECT:** Recuperación de datos.
- **INSERT:** Inserción de nuevos registros.
- **UPDATE:** Modificación de registros existentes.
- **DELETE:** Eliminación de datos.

La complejidad de SQL radica en su modularidad; una misma cláusula como **WHERE** puede ser compartida por tres de las cuatro instrucciones principales, lo que requiere una gramática BNF bien estructurada para evitar ambigüedades lógicas en el autómata de pila.

Objetivos

Objetivo General

Construir un analizador léxico y sintáctico capaz de validar un subconjunto del lenguaje SQL que incluya las operaciones fundamentales de manipulación de datos (CRUD).

Objetivos Específicos

- Implementar un escáner en Lex que reconozca palabras clave de SQL de forma insensible a mayúsculas/minúsculas, así como identificadores, cadenas y operadores.
- Desarrollar una gramática en Yacc que estructure y valide las dependencias de las sentencias SELECT, INSERT, UPDATE y DELETE.
- Validar la detección y recuperación de errores en consultas SQL mal formadas.

Desarrollo

Librerías y Variables Externas

Para el funcionamiento del analizador y la redirección de flujos, se incluyeron las cabeceras estándar de C (`<stdio.h>`, `<stdlib.h>`) junto con `"y.tab.h"` para la sincronización de tokens. Se utilizó el puntero `extern FILE *yyin` para redirigir la lectura desde archivos externos con extensión `.sql`.

Compatibilidad de Formato (El Retorno de Windows)

Al igual que en prácticas anteriores, para asegurar la estabilidad multiplataforma del analizador (especialmente en entornos Windows), se implementó una regla de expresión regular en Lex (`\r { /* Ignorar */ }`) para consumir de forma silenciosa el retorno de carro insertado por el sistema operativo, evitando desfasajes lógicos en la evaluación sintáctica.

Código del Analizador Léxico (lexico.l)

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}
%option noyywrap
%option case-insensitive
%%
"SELECT"      { return SELECT; }
"FROM"        { return FROM; }
"WHERE"       { return WHERE; }
"AND"         { return AND; }
"OR"          { return OR; }
"INSERT"      { return INSERT; }
"INTO"        { return INTO; }
"VALUES"      { return VALUES; }
"UPDATE"      { return UPDATE; }
"SET"         { return SET; }
"DELETE"      { return DELETE; }

[a-zA-Z_][a-zA-Z0-9_]* { return IDENTIFICADOR; }
[0-9]+(\.[0-9]+)?      { return NUMERO; }
" " [^ '\n]* " " ----- { return CADENA; }

"=" ----- { return IGUAL; }
">" | "<" | ">=" | "<=" | "!=" ----- { return OPERADOR_COMP; }
"*" ----- { return ASTERISCO; }
"," ----- { return COMA; }
";" ----- { return PUNTO_COMA; }
"(" ----- { return PARENTESIS_IJQ; }
")" ----- { return PARENTESIS_DER; }

\n ----- { /* Ignorar saltos de línea */ }
\r ----- { /* Ignorar retorno de Windows */ }
[-\t]+ ----- { /* Ignorar espacios y tabulaciones */ }
. ----- { return ERROR_LEXICO; }
%%
```

Código del Analizador Sintáctico (sintactico.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
extern FILE *yyin;
extern int yylex();
```

```

void yyerror(const char *s);
int consulta_num = 1;
%}
%token SELECT FROM WHERE AND OR INSERT INTO VALUES UPDATE SET DELETE
%token IDENTIFICADOR NUMERO CADENA IGUAL OPERADOR_COMP ASTERISCO
%token COMA PUNTO_COMA PARENTESIS_IZQ PARENTESIS_DER ERROR_LEXICO
%%
consultas: | consultas consulta ;

consulta:
    instruccion PUNTO_COMA {
        printf("->-EXITOSO-( Consulta-%d): -Sentencia -SQL- valida.\n",
            consulta_num++);
    }
    | error PUNTO_COMA {
        yyerrok;
        printf("->-ERROR-SINTACTICO-( Consulta-%d): -Consulta -SQL- mal
        -----formada.\n", consulta_num++);
    }
    ;

instruccion:
    instruccion_select | instruccion_insert |
    instruccion_update | instruccion_delete ;

instruccion_select:
    SELECT campos FROM tabla | SELECT campos FROM
    tabla WHERE condicion ;

campos: ASTERISCO | lista_identificadores ;

instruccion_insert:
    INSERT INTO tabla VALUES PARENTESIS_IZQ lista_valores
    PARENTESIS_DER
    | INSERT INTO tabla PARENTESIS_IZQ lista_identificadores
    PARENTESIS_DER VALUES PARENTESIS_IZQ lista_valores PARENTESIS_DER
    ;

instruccion_update:
    UPDATE tabla SET lista_asignaciones | UPDATE tabla
    SET lista_asignaciones WHERE condicion ;

lista_asignaciones:
    asignacion | lista_asignaciones COMA asignacion ;

asignacion: IDENTIFICADOR IGUAL valor ;

instruccion_delete:
    DELETE FROM tabla | DELETE FROM tabla WHERE condicion ;

tabla: IDENTIFICADOR ;

lista_identificadores:
    IDENTIFICADOR | lista_identificadores COMA IDENTIFICADOR ;

```

```

lista_valores:
valor | lista_valores COMA valor ;

condicion:
expresion | condicion AND expresion | condicion OR expresion ;

expresion:
IDENTIFICADOR IGUAL valor | IDENTIFICADOR OPERADOR_COMP valor ;

valor: NUMERO | CADENA | IDENTIFICADOR ;
%%
void yyerror(const char *s) {}
int main(int argc, char **argv) {
    if (argc > 1) { yyin = fopen(argv[1], "r"); }
    printf("---INICIANDO MOTOR SINTACTICO SQL (CRUD) ---\n");
    yyparse();
    printf("---ANALISIS FINALIZADO---\n");
    return 0;
}

```

Resultados

El analizador fue sometido a pruebas con sentencias de los cuatro tipos (CRUD) para verificar la robustez de las reglas de enrutamiento definidas en Yacc.

Archivo correcto.sql:

```

SELECT nombre, stock FROM inventario WHERE id = 5;
INSERT INTO personal (nombre, turno) VALUES ('Juan', 'Matutino');
UPDATE sucursales SET activo = 'NO' WHERE ventas < 1000;
DELETE FROM evaluaciones WHERE dificultad > 8;

```

La consola indicó el éxito de las 4 operaciones, validando las sentencias sin encontrar ambigüedades lógicas.

```

C:\Flex Windows\EditPlusPortable\Lex\NewAnlSnSQL>sql_parser.exe correcto.txt
--- INICIANDO MOTOR SINTACTICO SQL (CRUD) ---
-> EXITOSO (Consulta 1): Sentencia SQL valida.
-> EXITOSO (Consulta 2): Sentencia SQL valida.
-> EXITOSO (Consulta 3): Sentencia SQL valida.
-> EXITOSO (Consulta 4): Sentencia SQL valida.
--- ANALISIS FINALIZADO ---

```

Figure 1: Ejecución del programa.

Archivo incorrecto.sql:

```

INSERT personal VALUES ('Ana', 'Vespertino');
UPDATE sucursales activo = 'NO' WHERE ventas < 1000;
DELETE FROM evaluaciones WHERE;

```

El parser rechazó satisfactoriamente las consultas al detectar la falta de la palabra reservada INTO, la omisión de SET en la actualización y la carencia de una condición lógica tras el bloque WHERE.

```
C:\Flex Windows\EditPlusPortable\Lex\NewAnlSnSQL>sql_parser.exe incorrecto.txt
--- INICIANDO MOTOR SINTACTICO SQL (CRUD) ---
-> ERROR SINTACTICO (Consulta 1): Consulta SQL mal formada.
-> ERROR SINTACTICO (Consulta 2): Consulta SQL mal formada.
-> ERROR SINTACTICO (Consulta 3): Consulta SQL mal formada.
--- ANALISIS FINALIZADO ---
```

Figure 2: Ejecución del programa.

Conclusiones

La implementación de un mini motor sintáctico para SQL demuestra la versatilidad de la notación BNF y de la herramienta Yacc para manejar el enrutamiento de múltiples tipos de instrucciones (Create, Read, Update, Delete) partiendo de un mismo bloque inicial. Reutilizar símbolos no terminales como `condicion` o `lista_identificadores` entre las diferentes operaciones simplificó enormemente el diseño y evitó la redundancia algorítmica.

Cuestionario Semántico

1. ¿Qué es el análisis semántico y cómo se diferencia del sintáctico?

El análisis semántico verifica la coherencia lógica y de significado de una instrucción validada, mientras que el sintáctico únicamente verifica la estructura y el orden gramatical de los tokens basándose en las reglas de producción.

2. ¿Para qué sirve la Tabla de Símbolos en un compilador?

Sirve como un diccionario central en memoria para registrar y rastrear el contexto de todo el programa, guardando vinculaciones clave como nombres de variables, tipos de datos y los ámbitos o scopes lógicos correspondientes.

3. ¿Qué es la comprobación de tipos (Type Checking)?

Es una técnica central del análisis semántico enfocada en asegurar que las operaciones (matemáticas o de asignación) se realicen estrictamente entre estructuras de datos compatibles.

Bibliografía

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). Compilers: Principles, Techniques, and Tools. Pearson Education.
- Cooper, K. D., & Torczon, L. (2011). Engineering a Compiler. Morgan Kaufmann.